

SQL Injection UNION Attack: Determining the Number of Columns Returned

1. Vulnerability Overview

- **Vulnerability Type:** SQL Injection — UNION Attack / Column Enumeration
- **Security Impact:** Database Query Manipulation
- **Primary Objective:** Determine the exact number of columns returned by the application's original SQL query.

2. Vulnerability Description

A successful **UNION**-based SQL injection requires the injected query to be compatible with the original query in terms of column count and, depending on the database engine, compatible data types.

An attacker can therefore use controlled SQL manipulation to determine the structure of the original query.

Common assessment techniques include testing different **ORDER BY** positions or incrementally introducing **NULL** values through **UNION SELECT** statements until the application produces a valid response.

3. Attack Vector & Exploitation Methodology

The assessment identifies an input parameter that influences a backend SQL query.

The tester then systematically modifies the parameter to determine how many columns are returned by the original query.

Successful query execution without a database error provides an indication of the required column count for subsequent UNION-based testing.

4. Security Impact

Although column enumeration alone may not directly expose sensitive information, it can provide critical information required for further SQL injection exploitation.

Potential consequences include:

- Identification of the underlying query structure.
- Preparation for UNION-based data extraction.
- Increased ability to enumerate database contents.
- Potential exposure of sensitive application data through subsequent attacks.

5. Root Cause

The root cause is insufficient separation between user-controlled input and SQL query construction.

The application allows externally supplied input to influence the structure of the database query.

6. Remediation

- Implement **parameterized queries / prepared statements**.
- Avoid dynamic SQL construction using untrusted input.
- Apply strict server-side input validation.
- Use safe database-access abstractions where appropriate.
- Restrict database privileges to minimize the impact of successful exploitation.

7. Security Recommendation

All application-controlled database queries should use parameterized interfaces so that user input cannot alter the structure, operators, or logical behavior of the underlying SQL statement.

Author: Moe Htet Ar Kar (Phoe Cho)